JavaScript Interview Questions

Core language, closures, async/event-loop, and machine-coding problems

Compiled from real interview experiences (GeeksforGeeks, Zalon, GrowthGear, CREATE Bytes, DigiStay, Coding Blocks) + curated additions

---

Topics covered: var/let/const & hoisting · closures & lexical scope · this/call/apply/bind · debounce & throttle · event loop & microtasks · prototypes & OOP · generators & async · browser storage · array/object internals · machine coding
Difficulty split: Easy → Medium → Hard, each with theory, output-based, and coding questions

Format: Every section is split into **Easy** → **Medium** → **Hard**, and spans theoretical concepts, output-based/tricky code, and practical / machine-coding style problems.

EASY LEVEL

var vs let vs const

1. Explain the differences between var, let, and const in terms of scope, hoisting, and re-assignment.

2. Why is const not truly immutable for objects and arrays? Give an example.

3. What is block scope vs function scope? Which keywords respect which?

Hoisting

4. What is hoisting in JavaScript? Does it apply to var, let, const, and function declarations differently?

5. Predict the output:

```
console.log(a);
var a = 10;
```

■ *undefined (declaration hoisted, initialization not)*

6. Predict the output:

```
console.log(b);
let b = 10;
```

■ *ReferenceError - Temporal Dead Zone*

7. Are function declarations hoisted with their body? Are function expressions?

null, undefined & typeof

8. Explain the difference between null, undefined, and 'not defined' with examples of each.

9. What does typeof null return, and why is this considered a historic JS bug?

10. What is the output of: console.log(null == undefined) and console.log(null === undefined)? Explain why.

11. What is the output of: console.log(typeof undeclaredVar)?

■ *'undefined' — typeof never throws for undeclared identifiers*

JSON basics

12. Predict the output of each:

```
JSON.stringify({a: undefined})
JSON.stringify({a: null})
JSON.stringify([null])
JSON.stringify([undefined])
```

■ *'{}' , '{"a":null}' , '[null]' , '[null]' — undefined keys are dropped in objects but become null inside arrays*

13. What is the difference between JSON.parse and JSON.stringify? Give a real use case for each.

14. Does 'JSON.toObject' exist as a native method? What is the correct way to convert a JSON string to an object?

Data types & operators

15. List all primitive data types in JavaScript.

16. What is the difference between == and ===? When (if ever) is == acceptable to use?

17. What is NaN? Why does NaN === NaN evaluate to false, and how do you correctly check for NaN?

18. What is the output of: console.log(0.1 + 0.2 === 0.3)? Why does this happen and how would you compare floats safely?

■ *false, due to IEEE-754 floating point precision; compare with Math.abs(a-b) < Number.EPSILON*

Functions basics

19. What is the difference between a function declaration and a function expression?

20. What is an arrow function? List 2 key differences between arrow functions and regular functions.

21. What is a default parameter? Give an example.

22. What is the rest parameter and how is it different from the arguments object?

Basic DOM/events

23. What is event bubbling in JavaScript?

24. What is the difference between inline, block, and inline-block elements? (conceptual bridge to CSS)

25. What is the difference between addEventListener and using an inline onclick attribute?

MEDIUM LEVEL

Closures & Lexical Scope

1. What is a closure? Explain with a real-world example (e.g. a counter factory).

2. Explain lexical scope vs lexical environment with an example.

3. Predict the output and explain why:

```
function helper(fn) {
  let a = 3;
  return fn;
}
const ans = helper(() => {
  console.log(a);
});
ans();
```

■ *ReferenceError: a is not defined — closures capture lexical scope of where the function was DEFINED, not where it is called from*

4. Predict the output (var vs let counter pattern):

```
function counter(){
  var count = 0;
  return () => count++;
}
console.log(counter()());
console.log(counter()());
let c = counter();
console.log(c());
console.log(c());
```

■ *0, 0, 0, 1 — each counter() call creates a fresh closure; only 'c' reuses the same closure across calls*

5. Why do closures inside loops with var behave unexpectedly? How does let fix it?

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 0);
}
```

■ *First loop logs 3,3,3 (shared var i); second logs 0,1,2 (new binding of j per iteration)*

this, call/apply/bind

6. Explain call(), apply(), and bind() with a real-life use case for each.

7. How does the value of 'this' get determined in a regular function vs an arrow function?

8. Predict the output:

```
const obj = {
  name: 'Alice',
  sayHello: function () {
    const inner = () => {
      console.log(this.name);
    };
    inner();
  }
};
obj.sayHello();
```

■ *'Alice' — arrow functions don't have their own 'this'; they inherit it from the enclosing (sayHello) scope*

9. Implement your own version of Function.prototype.bind from scratch.

Debouncing & Throttling

10. What is debouncing? Implement a debounce() utility function from scratch in JavaScript.

11. What is throttling? Implement a throttle() utility function from scratch.

12. Explain the difference between debouncing and throttling with a real use case for each (e.g. search-as-you-type vs scroll listener).

13. How would debounce and throttle utilities need to correctly preserve 'this' and arguments using call/apply?

Browser Storage

14. Compare localStorage, sessionStorage, and cookies across: persistence, size limit, sent-with-requests, accessibility from JS.

15. When would you use cookies over localStorage for storing an auth token? What are the security implications (httpOnly, secure, SameSite)?

16. What are the advantages and disadvantages of each of the three storage types with a real example scenario for each?

Event Loop

17. Explain the JavaScript event loop, call stack, microtask queue, and macrotask (callback) queue.

18. Predict the output and explain the ordering:

```
console.log('start');
setTimeout(() => {
  console.log('timeout');
}, 0);
Promise.resolve().then(() => {
  console.log('promise');
});
console.log('end');
```

■ *start, end, promise, timeout — microtasks (Promises) always run before the next macrotask (setTimeout)*

19. Are Promise .then() callbacks micro or macro tasks? Where does queueMicrotask fit in?

use strict & TDZ

20. What does 'use strict' do? Name 3 concrete behavior changes it introduces.

21. What is the Temporal Dead Zone (TDZ)? Which declarations are affected?

22. Predict the output:

```
console.log(typeof a);
let a = 5;
```

■ *ReferenceError — 'typeof' on a TDZ variable still throws, unlike a truly undeclared variable*

Objects, spread & references

23. Predict the output and explain reference vs value semantics:

```
const a = { x: 1 };
const b = a;
b.x = 2;
console.log(a.x);

const c = { x: 1 };
const d = { ...c };
d.x = 2;
console.log(c.x);
```

■ *2, then 1 — object assignment copies the reference; spread creates a shallow copy*

24. What is the difference between a shallow copy and a deep copy? How would you deep clone a nested object?

25. Given an object with a computed key, predict the output:

```
let a = 3;
let b = { x: 'def' };
b = { [a]: b.x };
console.log(b);
```

■ *{ 3: 'def' } — computed property names evaluate the expression as the key*

26. How would you deeply update a nested object property immutably (without mutating the original)?

Array & String methods

27. Predict the output:

```
let arr = [1, 2, 3, 4, 5];
arr.splice(2, 2, 6, 7);
console.log(arr);
```

■ *[1, 2, 6, 7, 5] — removes 2 elements from index 2 and inserts 6, 7*

28. Implement your own polyfills for Array.prototype.map, filter, and reduce.

29. Sort a mixed-type numeric-string array in both ascending and descending order:

```
let arr = ['12', '2', '22', '56', '2'];
// sort ascending & descending
```

30. How do array prototype methods like forEach, map, and reduce differ in return value and use case?

Prototypes & OOP in JS

31. Explain prototypal inheritance in JavaScript. How is it different from classical (class-based) inheritance?

32. What is the prototype chain? How does property lookup work when a property is not found on an object?

33. What is the difference between Object.create(), a constructor function, and the class syntax for creating objects?

34. Explain higher-order functions with a real-world example.

CSS-adjacent front-end

35. Explain position: absolute, relative, sticky, and fixed with the differences between each.

36. Explain CSS preprocessors (SCSS/SASS) and CSS variables. What problems do they solve over plain CSS?

37. How would you implement a smooth left-to-right CSS transition/animation on hover?

HARD LEVEL

Advanced Closures / Scoping output-based

1. Predict the output and explain WHY the two counters behave differently:

```
function counter(){
  let count = 0;
  return () => count++;
}
console.log(counter()());
console.log(counter()());
console.log(counter()());
let c = counter();
console.log(c());
console.log(c());
console.log(c());
```

■ *0,0,0 then 0,1,2 — first three calls create a NEW closure each time; 'c' reuses one closure so count increments across calls*

2. Implement a memoize() higher-order function that caches results of a pure function based on its arguments.

3. Implement a curry() function that transforms f(a,b,c) into f(a)(b)(c) and also supports f(a,b)(c) and f(a,b,c).

4. Implement a simple pub-sub / event emitter class from scratch in plain JavaScript (on, off, emit).

Async, Generators & Event Loop deep dive

5. Explain generators in JavaScript. What problem do they solve, and how do they relate to async/await under the hood?

6. Predict the output:

```
function* generatorFn() {
  yield 1;
  yield 2;
  yield 3;
}
const generator = generatorFn();
console.log(generator.next().value);
console.log(generator.next().value);
console.log(generator.next().value);
```

■ *1, 2, 3*

7. Implement your own simplified version of Promise.all() using native Promises.

8. Explain how async/await is essentially syntactic sugar over Promises and the microtask queue. What happens internally when you 'await' a value?

9. Write a function that resolves an array of async tasks with a concurrency limit (e.g. only 3 running at a time) — a common machine coding round question.

Machine Coding — Practical Problems

10. Fraud Detection System: Given an array of transactions {userId, amount, timestamp}, write a function to detect userIds that made 3+ transactions within a 600-second window where amounts are within 10% of each other.

```
const transactions = [
  { userId: 1, amount: 100, timestamp: 1690000100 },
  { userId: 2, amount: 200, timestamp: 1690000150 },
  { userId: 1, amount: 105, timestamp: 1690000200 },
  { userId: 1, amount: 98,  timestamp: 1690000250 },
  { userId: 2, amount: 210, timestamp: 1690000850 },
  { userId: 1, amount: 110, timestamp: 1690000500 },
  { userId: 3, amount: 300, timestamp: 1690000600 },
  { userId: 2, amount: 250, timestamp: 1690000150 },
  { userId: 2, amount: 210, timestamp: 1690000200 },
];
```

■ *Group by userId, sort by timestamp, use a sliding window checking both the time span and amount similarity ratio*

11. Optimize the following React-adjacent JS snippet — identify all the bugs/inefficiencies (missing dependency array, mutation, missing key, no loading/empty state):

```
function UserList({ users }) {
  const [filteredUsers, setFilteredUsers] = useState([]);
  useEffect(() => {
    setFilteredUsers(users.filter(user => user.active));
  });
  return (
    <div>
      {filteredUsers.map(user => (
        <div onClick={() => console.log(user.id)}>
          {user.name}
        </div>
      ))}
    </div>
  );
}
```

12. Given three strings, determine whether the third string is an interleaving of the first two (order-preserving merge), e.g. s1='abc', s2='def', s3='adbcfe' → true, s3='adbcfe' altered → false.

```
s1 = 'aabcc', s2 = 'dbbca', s3 = 'aadbbcbcac'  // true (interleaving)
```

13. Implement a rate limiter (token bucket or sliding window) in plain JavaScript that limits a function to N calls per T milliseconds.

14. Implement a deep-freeze function that recursively makes a nested object fully immutable.

15. Build a simple in-memory LRU cache class with get(key) and put(key, value), each O(1).

Module system & internals

16. Explain the module caching behavior: if a.js is required twice from b.js, is its top-level code executed once or twice? Why?

```
// a.js
console.log('a.js loaded');
module.exports = {};

// b.js
require('./a');
require('./a');
```

■ *'a.js loaded' prints ONCE — Node/CommonJS caches modules by resolved file path after the first require*

17. Explain the difference between CommonJS (require/module.exports) and ES Modules (import/export). Can you mix them, and what are the pitfalls?

18. What is a WeakMap and WeakSet? Give a real use case where they prevent memory leaks compared to Map/Set.

19. Explain how JavaScript's garbage collector identifies unreachable objects (mark-and-sweep) and how closures can accidentally cause memory leaks.

Tricky output-based (advanced)

20. Predict the output and explain:

```
let x = 10;
function foo() {
  x = 20;
  console.log(x);
}
foo();
console.log(x);
```

■ *20, 20 — no 'var/let' inside foo means it mutates the outer x (no shadowing)*

21. Predict the output:

```
let result = null == undefined;
console.log(result);
```

■ *true — loose equality treats null and undefined as equal to each other (but not to anything else)*

22. Explain what happens step by step (call stack + task queues) for a mixed Promise/setTimeout/sync code block with nested .then() chains — draw out the execution order.

23. Why can array holes (sparse arrays) behave differently with forEach vs map vs a for loop? Give an example.